

Experiment7

- 1) Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

Code :

```
#include <stdio.h>
#include <stdlib.h>

int global_var = 10; // Global variable

void code_function()
{   printf("Code address  : %p\n", (void
*)code_function);
}

int main() {   static int static_var = 20; //
Static variable   int stack_var = 30;      //
Stack variable
    int *heap_var = malloc(sizeof(int)); // Heap variable

    if (heap_var == NULL)
    {
        printf("Memory allocation failed\n");
return 1;
    }

    *heap_var = 40;

    printf("Linux Process Address Space\n");   printf("----
-----\n");

    code_function();

    printf("Global address : %p\n", (void *)&global_var);
    printf("Static address : %p\n", (void *)&static_var);
    printf("Heap address   : %p\n", (void *)heap_var);
    printf("Stack address  : %p\n", (void *)&stack_var);
```

```

        free(heap_var);

    return 0;
}

```

OUTPUT :

```

teja27@TEJA:~$ nano exp7.c
teja27@TEJA:~$ gcc exp7.c -o exp7
teja27@TEJA:~$ ./exp7
Linux Process Address Space
-----
Code address   : 0x643298c601c9
Global address : 0x643298c63010
Static address : 0x643298c63014
Heap address   : 0x6432d0fa9010
Stack address  : 0x7fff642e3dac
teja27@TEJA:~$ |

```

2) Make use of the /proc//maps file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

Code:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
int global_var = 100;
```

```
int main()
```

```
{
```

```
    static int static_var = 200;
```

```
int stack_var = 300;
```

```
int *heap_var = malloc(sizeof(int));
```

```
*heap_var = 400;

printf("Process ID (PID): %d\n", getpid());

printf("Global variable address : %p\n", (void *)&global_var);

printf("Static variable address : %p\n", (void *)&static_var);

printf("Heap variable address : %p\n", (void *)heap_var);

printf("Stack variable address : %p\n", (void *)&stack_var);

printf("\nProcess is running...\n");   getchar();

free(heap_var); return 0;

}
```

OUTPUT:

```
teja27@TEJA:~$ nano exp7.1.c
teja27@TEJA:~$ gcc exp7.1.c -o exp7.1
teja27@TEJA:~$ ./exp7.1
Process ID (PID): 559
Global variable address : 0x61b04f239010
Static variable address : 0x61b04f239014
Heap variable address   : 0x61b066fdf010
Stack variable address  : 0x7ffea19da3cc

Process is running...
|
```